Travaux pratiques Analyse en composantes principales

- Cas d’école « théorique »
- ACP appliquée aux cellules
- ACP appliquée aux Vins
- ACP pour la reconnaissance des Visages

L'objectif de cette séance de TP est de présenter l'utilisation des fonctionnalités de Scikit-learn concernant l'ACP, ainsi que de contribuer à une meilleure compréhension de cette méthode d'analyse de données. Après une brève présentation de la classe correspondante de Scikit-learn, nous examinons d'abord des données générées de façon contrôlée afin de mieux comprendre l'impact de la distribution des données sur les résultats de l'ACP. Nous appliquons ensuite l'ACP à un jeu standard et limité de données réelles : les vins. En fin nous ferons de même avec un jeu de données à grande dimension : la base de visages Olivetti, où chaque individu est une image de visage décrite par 4096 variables (pixels), pour 40 personnes. Cette situation, où le nombre de variables dépasse très largement le nombre d'individus, illustre l'usage de l'ACP comme outil de compression (les « visages propres ») et comme prétraitement d'un classifieur par machine à vecteurs de support (SVM).

On reprendra avec profit les fichiers sources vus en CM et TD mais évidemment, copier/coller sans vraiment lire le code ne sert pas à grand-chose : on n’hésitera donc pas à faire ses propres tests et affichages pour s’approprier les notions.

On pourra, s’il on l’a installé, utiliser JUPYTER afin de mettre à la fois son code et à la fois ses réponses aux questions.

L'ACP avec PCA dans Scikit-learn

Dans Scikit-learn, l'analyse en composantes principales est mise en œuvre dans la classe `sklearn.decomposition.PCA` (voir https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html). Pour définir l'analyse on appelle PCA() ou `PCA(n_components=None, copy=True, whiten=False, svd_solver='auto', tol=0.0, iterated_power='auto', random_state=None)`.

Les paramètres d'appel sont similaires à la plupart des méthodes `fit` de Scikit-learn :

- `n_components` : nombre de composantes à conserver (par défaut toutes, c'est-à-dire `min(n_samples, n_features)`). Il est possible de donner un entier (nombre d'axes) ou une valeur réelle entre 0 et 1 (fraction de variance à conserver, par exemple `PCA(n_components=0.95)`).
- `copy` : si `False`, la matrice de données est écrasée par les données transformées (par défaut `True`).
- `whiten` : si `True`, les données transformées sont modifiées pour que les projections sur les axes principaux présentent une variance unitaire (par défaut `False`).
- `svd_solver` : solveur à employer parmi `'auto'`, `'full'`, `'arpack'`, `'randomized'` (par défaut `'auto'`).
- `tol`, `iterated_power`, `random_state` : paramètres optionnels pour solveurs spécifiques.

Les attributs accessibles sont les suivants :

- `explained_variance_` : array `[n_components]` dans lequel on trouve les valeurs propres (la variance normalisée) en ordre décroissant.
- `components_` : array `[n_components, n_features]` dans lequel chaque ligne correspond à un vecteur propre ; l'ordre est celui des valeurs propres.
- `explained_variance_ratio_` : array `[n_components]` avec les valeurs propres exprimées en pourcentage de la variance totale.
- `mean_` : array `[n_features]` contenant les moyennes des variables calculées lors de l'ACP.
- `n_components_` : nombre estimé de composantes.
- `noise_variance_` : variance du bruit estimée suivant l'approche de l'ACP probabiliste.

Les méthodes qui peuvent être employées :

- `fit(X[, y])` : trouver les axes principaux et les valeurs propres associés à partir de la matrice de données X.
- `fit_transform(X[, y])` : trouver les axes principaux et les valeurs propres, puis appliquer la projection sur les axes principaux.
- `transform(X[, y])` : appliquer la projection sur les axes principaux (l'ACP doit avoir été estimée avant par `fit`).
- `inverse_transform(X[, y])` : re-projeter les données transformées dans l'espace initial.
- `get_covariance()` : calculer les covariances à partir de la matrice de données.
- `get_params([deep])`, `set_params(**params)` : lire et modifier les paramètres de l'estimateur.
- `score(X[, y])`, `score_samples(X)` : log-vraisemblance (ACP probabiliste).

ACP sur des données générées mathématiquement

L’idée de ce TP est d’appliquer la même ACP à deux nuages de points aléatoires puis de refaire de même sur ces deux nuages de points déformés dans une certaine direction.

Démarrez par la génération d'un ensemble de 500 vecteurs dans l'espace 3D, suivant une loi normale (de moyenne nulle et de variance unitaire), ensuite visualisez les points générés :

```
import numpy as np
import matplotlib.pyplot as plt

# Génération de données selon une loi normale tridimensionnelle
mesVectAlea = np.random.randn(500, 3)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(mesVectAlea[:,0], mesVectAlea[:,1], mesVectAlea[:,2])
plt.title("Données initiales")
plt.show()
```

Appliquez ensuite l'ACP à ces données :

```
from sklearn.decomposition import PCA

monACP = PCA(n_components=3)
monACP.fit(mesVectAlea)

print("Pourcentage de variance expliquée : ",monACP.explained_variance_ratio_)
print("Composantes principales : ")
print(monACP.components_)
```

Chaque ligne de `pca.components_` représente le vecteur unitaire qui donne la direction d’un axe factoriel. Le vecteur `pca.components_[i,:]` correspond à la valeur propre `pca.explained_variance_ratio_[i]` (la variance portée).

ACP avec un nouveau tableau

Générez un autre tableau de données en utilisant de nouveau `np.random.randn(500,3)` et appliquez l'ACP à ces nouvelles données. Comparez les résultats avec ceux obtenus sur `précédemment`. Que constatez-vous ? Expliquez.

Correction

On constate d'abord que les trois valeurs propres sont chaque fois très proches entre elles, ce qui indique que le nuage d'observations a une forme relativement sphérique. Ensuite, en examinant les vecteurs propres correspondants, on constate que d'un ensemble de données générées à un autre les directions des vecteurs propres changent ; ces vecteurs n'ont donc pas de pertinence particulière pour décrire les données, ce qui est cohérent avec les faibles différences entre valeurs propres successives.

ACP sur des données tournées et agrandies dans certaines directions

Appliquez maintenant une déformation et une rotation dans l'espace tridimensionnel au nuage des observations de `mesVectAlea` :

```
s1 = np.array([[3,0,0], [0,1,0], [0,0,0.2]])  # matrice de déformation
r1 = np.array([[0.36,0.48,-0.8], [-0.8,0.6,0], [0.48,0.64,0.6]])  # matrice de rotation
mesVectDeformes = mesVectAlea.dot(s1).dot(r1)
```

Et appliquez de nouveau une ACP à mesVectDeformes.

Que constatez-vous par rapport à l'application directe de l'ACP sur les données de `mesVectAlea` ? Expliquez. Peut-on lier les rapports entre les valeurs propres obtenues aux rapports entre les éléments de la diagonale de la matrice de déformation `s1` ?

Correction

La décroissance des valeurs propres est ici très rapide : le premier axe factoriel explique environ 90 % de la variance. Les rapports entre valeurs propres correspondent aux carrés des rapports entre les éléments de la diagonale de `s1`. En effet, les valeurs propres sont des variances alors que les éléments de la diagonale de la matrice de déformation ont ici une signification d'écart-types.

ACP sur de nouvelles données transformées à l’identique

Générez un autre ensemble de données en utilisant les mêmes matrices de transformation et appliquez l'ACP à ces nouvelles données. Comparez les résultats avec ceux obtenus sur `mesVectDeformes`. Que constatez-vous ? Expliquez.

Correction

Les valeurs propres obtenues sur le second ensemble de données sont très proches de celles obtenues sur `rndef`. De plus, entre les deux ensembles de données, les vecteurs propres correspondant aux valeurs propres de même ordre sont très similaires. Les valeurs et vecteurs propres caractérisent les (fortes) déformations appliquées aux nuages d'observations générés et sont donc relativement stables par rapport à un changement d'échantillon.

Les données « cellules »

Présentation

Les données utilisées dans la suite sont issues de l'ensemble *Breast Cancer Wisconsin (Diagnostic)*, disponible directement dans Scikit-learn. Chaque observation décrit une cellule (noyau cellulaire, plus précisément) par 30 mesures morphologiques (rayon, texture, périmètre, aire, régularité, concavité, etc., chacune déclinée en moyenne, écart-type et valeur maximale sur l'image du noyau). La variable à expliquer `y` vaut 0 pour une cellule maligne et 1 pour une cellule bénigne. L'ensemble compte 569 observations et 30 variables, toutes quantitatives.

```
from sklearn import preprocessing
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()
X = cancer.data     # (569, 30)
y = cancer.target   # (569,) : 0 = maligne, 1 = bénigne
coul = ["red" if yy == 0 else "blue" for yy in y]
nomsVars = cancer.feature_names

print("Dimensions du tableau :", X.shape)
print("Effectifs par valeur de y :", [y.tolist().count(k) for k in range(2)])
print(f"{len(nomsVars)} variables sur {len(y)} cellules.")
```

Chaque cellule est donc décrite dans un espace à 30 dimensions : une visualisation directe est impossible, et c'est précisément le rôle de l'ACP de produire une représentation en 2 ou 3 dimensions qui conserve le mieux possible l'inertie du nuage.

Centrage, réduction et calcul de l'ACP

Les variables de cet ensemble sont hétérogènes : les aires ont des valeurs en centaines ou milliers alors que certaines régularités sont proches de 0,1. Que risque-t-il de se produire si l'on applique l'ACP directement sur `X` sans transformation préalable ? Vérifiez expérimentalement.

```
pcaBrut = PCA().fit(X)
print(pcaBrut.explained_variance_ratio_[:5])
```

Correction

Sans réduction, les variables à forte variance dominent très largement dans la définition de la direction du premier axe factoriel. La première valeur propre explique alors la quasi-totalité de l'inertie, sans que cela reflète une structure réelle des données.

Calcul (entraînement) de l’ACP après réduction

Comme pour les données sur le sommeil des mammifères vues en cours, l'ACP directe n'est pas une ACP normée : il faut centrer et réduire les variables pour qu'elles aient toutes le même poids dans l'analyse.

L'entraînement de l'ACP, c'est-à-dire le calcul des axes factoriels et des valeurs propres par Scikit-learn, se fait classiquement avec `fit` :

```
# cellulesCR est le tableau NumPy des mesures centrées réduites
cellulesCR = preprocessing.scale(X)


monACP = PCA().fit(cellulesCR)
# ou PCA(n_components=2) pour limiter le nombre de composantes,
# ou PCA(n_components=0.95) pour conserver 95 % de la variance
```

On peut ensuite afficher les premiers résultats du calcul selon la manière habituelle :

```
print("\nVariance expliquée par axe :")
for i, (vp, part) in enumerate(zip(monACP.explained_variance_, monACP.explained_variance_ratio_)):
    print(f"  Axe {i+1} : valeur propre = {vp:6.3f} | part = {part:6.2%} "
          f"| cumul = {monACP.explained_variance_ratio_[:i].sum():6.2%}")

titre = '1 Pourcentage d’inertie associé à chaque axe factoriel'
seuil = 0.95
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111)
indices = range(len(monACP.explained_variance_ratio_))
ax.bar(indices, monACP.explained_variance_ratio_, color="steelblue", label="Variance
expliquée")
ax.plot(indices, monACP.explained_variance_ratio_.cumsum(), "o-", color="crimson",
label="Variance cumulée")
ax.axhline(seuil, ls="--", color="grey", label=f"Seuil {int(seuil*100)} %")
ax.set_xticks(indices)
ax.set_xticklabels([f"PC{i+1}" for i in indices])
ax.set_xlabel("Composante principale")
ax.set_ylabel("% de variance expliquée")
ax.set_title(titre)
ax.legend()
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

Combien d'axes faut-il conserver pour atteindre 95 % de la variance ? Affichez le graphique de décroissance des valeurs propres.

Correction

Après réduction, la décroissance est beaucoup plus progressive : le premier axe explique environ 44 % de l'inertie, le deuxième environ 19 %, et il faut environ 10 axes pour atteindre 95 % de variance cumulée. Les 2 premiers axes capturent un peu moins des deux tiers de l'information, ce qui suffit pour une exploration visuelle.

Projection des individus (les cellules) sur deux axes factoriels

Les coordonnées des individus dans le repère formé par les axes factoriels s'obtiennent par la méthode `transform` :

```
cellulesDansNouveauRepere = monACP.transform(cellulesCR)
```

On peut afficher les coordonnées ligne par ligne (une ligne par cellule, une colonne par axe factoriel) :

```
print("Coordonnées des cellules dans le nouveau repère :")
print("                          ", end="")
for j in range(4):
    print(f" Axe {j+1} ", end=" ")
print()
for k, ligne in enumerate(cellulesDansNouveauRepere):
    print(f"  {k:4} :", end=" ")
    for valeur in ligne:
        print(f"{valeur:+.4f}", end=" ")
    print()
```

La légende des axes factoriels pour les graphiques intègre la part de variance de chaque axe :

```
legende_axes = [f"Axe {k+1} ({monACP.explained_variance_ratio_[k]:.1%})" for k in
range(4)]
```

On affiche ensuite le nuage des individus projeté sur les deux premiers axes factoriels, en colorant chaque cellule selon sa classe (rouge pour maligne, bleu pour bénigne). Les deux classes apparaissent-elles séparées ?

```
titre = "2 Le nuage des individus projeté sur les deux premiers axes"
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111)
ax.set_title("Cellules : " + titre)
for i in range(len(y)):
    xx, yy = cellulesDansNouveauRepere[i,0], cellulesDansNouveauRepere[i,1]
    ax.scatter(xx, yy, c=coul[i])
ax.set_xlabel(legende_axes[0])
ax.set_ylabel(legende_axes[1])
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

Correction

Les cellules bénignes se concentrent à droite du plan (coordonnées positives sur l'axe 1) et les malignes à gauche ; les deux classes apparaissent nettement séparées le long du premier axe factoriel, alors que l'axe 2 ne les discrimine pas. L'axe 1 peut donc être interprété comme un axe « agressivité » de la tumeur, construit par l'ACP sans jamais utiliser l'étiquette de classe.

Projection des individus (cellules) sur les deux axes suivants

Affichez le nuage des individus projeté sur les axes 3 et 4. Attention : il faut utiliser les colonnes d'indices 2 et 3 de `cellulesDansNouveauRepere`, au lieu des colonnes 0 et 1. Que constatez-vous par rapport à la séparation des classes ?

Correction

```
titre = "3 Le nuage des individus projeté sur les deux axes suivants"
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111)
ax.set_title("Cellules : " + titre)
for i in range(len(y)):
    xx, yy = cellulesDansNouveauRepere[i,2], cellulesDansNouveauRepere[i,3]
    ax.scatter(xx, yy, c=coul[i])
ax.set_xlabel(legende_axes[2])
ax.set_ylabel(legende_axes[3])
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

Sur ce plan, les deux classes sont fortement mélangées : les axes au-delà du deuxième n'opposent pas les cellules malignes et bénignes, ils décrivent d'autres sources de variabilité morphologique. La discrimination est concentrée sur les tout premiers axes, ce qui est cohérent avec la décroissance rapide de la variance expliquée.

Projection sur les trois premiers axes factoriels

Réalisez l'affichage des projections sur les 3 premiers axes principaux ; la possibilité de rotation du graphique 3D facilite la lecture du nuage. Cela apporte-il quelque chose

```
titre = "4 Le nuage des individus projeté sur les trois premiers axes"
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.set_title("Cellules : " + titre)
for i in range(len(y)):
 xx, yy, zz = cellulesDansNouveauRepere[i,0], cellulesDansNouveauRepere[i,1],
cellulesDansNouveauRepere[i,2]
 ax.scatter(xx, yy, zz, c=coul[i])
ax.set_xlabel(legende_axes[0])
ax.set_ylabel(legende_axes[1])
ax.set_zlabel(legende_axes[2])
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

Correction

Évidemment la projection est plus fidèle mais la difficulté de la vue 3D font que le gain n’est pas décisif.

Projection des variables sur les deux premiers axes factoriels

Question

Les résultats de l'analyse du nuage des variables ne sont pas directement disponibles dans Scikit-learn. Comment pouvez-vous les obtenir ? Affichez la projection du nuage des variables sur les deux premiers axes factoriels.

```
# projectionsVars : coordonnées de chacune des variables dans le nouveau repère
projectionsVars = monACP.components_.T * np.sqrt(monACP.explained_variance_)

titre = "5 Projection des variables sur les deux premiers axes factoriels"
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111)
ax.set_title("Cellules : " + titre)
ax.set_xlim([-1, 1]) ; ax.set_ylim([-1, 1])
ax.axhline(0, color='lightgray', lw=0.5) ; ax.axvline(0, color='lightgray', lw=0.5)
ax.add_artist(plt.Circle((0, 0), 1, color='b', fill=False))

for i, nom in enumerate(nomsVars):
    x, y = projectionsVars[i, 0], projectionsVars[i, 1]
    ax.arrow(0, 0, x, y, shape='full', lw=1, length_includes_head=True,
             head_width=0.03, head_length=0.05)
    ax.text(x, y, nom, fontsize=7)
ax.set_xlabel(legende_axes[0])
ax.set_ylabel(legende_axes[1])
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

Quelle est votre interprétation ? Pensez à trouver des éléments intéressants les biologistes.

Correction

Dans le cas d'une ACP normée, les coordonnées des variables dans le repère factoriel s'obtiennent en multipliant chaque vecteur propre par la racine carrée de la valeur propre correspondante (ce sont les corrélations entre variables initiales et composantes principales) :

Presque toutes les flèches pointent vers la droite du cercle : toutes les variables de taille et de forme sont fortement corrélées positivement au premier axe, qui synthétise un facteur « taille et irrégularité » du noyau. Les variables d'erreur-type (suffixe `_se`) forment un second groupe, aligné sur l'axe 2.

Projection des variables sur les 3 et 4

Affichez la projection des variables sur les axes 3 et 4.

Que constatez-vous par rapport aux projections sur les deux premiers axes ?

Correction

```
titre = "6 Projection des variables sur les deux derniers axes factoriels"
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111)
ax.set_title("Cellules : " + titre)
ax.set_xlim([-1, 1]) ; ax.set_ylim([-1, 1])
ax.axhline(0, color='lightgray', lw=0.5) ; ax.axvline(0, color='lightgray', lw=0.5)
ax.add_artist(plt.Circle((0, 0), 1, color='b', fill=False))

for i, nom in enumerate(nomsVars):
    x, y = projectionsVars[i, 2], projectionsVars[i, 3]
    ax.arrow(0, 0, x, y, shape='full', lw=1, length_includes_head=True,
             head_width=0.03, head_length=0.05)
    ax.text(x, y, nom, fontsize=7)
ax.set_xlabel(legende_axes[2])
ax.set_ylabel(legende_axes[3])
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

Les flèches sont plus courtes (proches du centre du cercle) : les variables sont faiblement corrélées aux axes 3 et 4, qui n'opposent pas de grands groupes de variables. Cela confirme que ces axes portent peu d'information discriminante et interprétable.

Vecteurs propres et interprétation des axes

Le tableau des vecteurs propres (lignes = variables initiales, colonnes = composantes principales) met en évidence la contribution de chaque variable à chaque axe :

```
composantes = [f"PC{i+1}" for i in range(monACP.n_components_)]

vecteurs = pd.DataFrame(
    monACP.components_.round(3),
    index=composantes,
    columns=nomsVars
).T  # transposé : lignes = variables, colonnes = composantes
print("=== Vecteurs propres (coordonnées par variable) ===")
print(vecteurs.to_string())
```

Examinez les coordonnées des variables sur le premier axe. Quelles variables contribuent le plus à l'axe 1 ? Cette lecture est-elle cohérente avec le cercle des corrélations et avec l'interprétation de la Question 6 ?

Correction

Sur le premier axe, toutes les variables de dimension (rayon, périmètre, aire) et d'irrégularité (points de concavité, concavité, fractale) présentent des coordonnées positives et de même ordre de grandeur, tandis que la texture y contribue moins. L'axe 1 oppose donc les noyaux de grande taille et irréguliers (cellules malignes, à gauche du plan des individus) aux noyaux petits et réguliers (cellules bénignes). Cette lecture est cohérente avec le cercle des corrélations, où toutes les flèches pointent du même côté sur l'axe 1.

Pour aller plus loin

Reconstitution

On utilise maintenant `PCA(n_components=0.95)` pour limiter automatiquement le nombre de composantes, puis `inverse_transform` pour re-projeter les données dans l'espace initial. Comparez, pour quelques cellules, les mesures initiales et les mesures reconstituées. Que constatez-vous ?

```
acp95 = PCA(n_components=0.95).fit(cellulesCR)
Z = acp95.transform(cellulesCR)
print("Nombre de composantes conservées :", acp95.n_components_)
Xrec = acp95.inverse_transform(Z)
print(cellulesCR[0, :5].round(3))
print(Xrec[0, :5].round(3))
```

Correction

Les mesures reconstituées sont proches des mesures initiales mais pas identiques : l'information portée par les derniers axes (environ 5 % de la variance) est perdue. Cette décomposition-recomposition est exactement le principe de la compression de données par ACP : conserver quelques dizaines de composantes suffit à approcher fidèlement les 569 observations.

Classification « à la louche » avec une simple ACP

L'ACP, appliquée ici sans utiliser l'étiquette de classe, a produit une représentation où les classes sont visiblement séparées. Que peut-on en conclure sur le potentiel d'un classifieur simple (par exemple, des plus proches voisins) entraîné sur les seules coordonnées factorielles des deux premiers axes ?

Correction

Puisque la séparation est nette dans le plan (PC1, PC2), un classifieur simple appliqué à ces deux coordonnées devrait obtenir une bonne performance, quoiqu'inférieure à celle obtenue sur les 30 variables complètes : quelques cellules malignes se projettent dans la zone des cellules bénignes. Mais a-t-on droit à l’erreur ?

La réduction de dimension par ACP conserve l'essentiel de l'information discriminante tout en divisant la dimension par quinze, ce qui réduit le coût calculatoire et le risque de surapprentissage. Il n’y a plus qu’à appliquer une méthode de classification sérieuse.

ACP sur les données Wine

Évidemment il n'est pas demandé d'apprécié ni même de connaître la moindre chose au vin. Il est justement tout à fait extraordinaire soit un outil qui nous permette de dire des choses intéressantes sur un sujet que l'on ne connaît pas !

Les données utilisées sont issues de l'ensemble *Wine Recognition* (Forina et al., Istituto di Farmacologia di Genova), issu d'une étude sur la cartographie chimique des vins italiens et distribué avec Scikit-learn. Chaque observation décrit un vin par 13 analyses physico-chimiques : degré alcoolique (`alcohol`), acidité malique (`malic_acid`), cendres (`ash`), alcalinité des cendres (`alcalinity_of_ash`), magnésium (`magnesium`), phénols totaux (`total_phenols`), flavonoïdes (`flavanoids`), phénols non flavonoïdes (`nonflavanoid_phenols`), proanthocyanidines (`proanthocyanins`), intensité de couleur (`color_intensity`), teinte (`hue`), indice OD280/OD315 des vins dilués (`od280/od315_of_diluted_wines`) et proline (`proline`).

Trois cépages sont représentés : Barolo (classe 0), Grignolino (classe 1) et Barbera (classe 2). L'ensemble compte 178 observations.

Cela n'est donné qu'à titre indicatif, aucune connaissance de ce type n'est utile dans ce TP.

Enregistrez une copie de votre script cellule en le nommant « wine » et remplacer l'intro par :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import preprocessing
from sklearn.decomposition import PCA
from sklearn.datasets import load_wine

vins = load_wine()
```

Regarder maintenant les attibuts `data`, `target` et `feature_names` de `vins` d'un point de vue « données » et « structures ». Décrivez-les : A quoi peuvent-ils nous servir ?

Correction

Vins.data est un tableau de valeurs numériques de 178 lignes et 13 colonnes. vins.feature_names est le nom de ces variables colonnes et vins.target le numéro du cépage qui pourra donc nous servir à colorer nos points.

Note : Les noms des cépages sont : 0 = Barolo, 1 = Grignolino, 2 = Barbera

Question classique

Combien de composantes principales l'ACP peut-elle au plus produire sur ces données ?

Correction

Jusqu'à 13 composantes puisque l'on a un nombre d'individus largement suffisant.

Calcul de l'ACP : centrage et réduction préalable

Les variables de cet ensemble sont-elles à échelles comparables ? Que risque-t-il de se produire si l'on applique l'ACP directement sur `X` sans réduction préalable ? Vérifiez expérimentalement.

Correction

Les échelles sont ici radicalement hétérogènes : la proline se compte en milliers (de l'ordre de 500 à 1700), le magnésium en dizaines, la plupart des indices entre 0 et 10, la teinte autour de 1. L'ACP directe n'est pas une ACP normée : la variance de la proline, de l'ordre du million, écrase toutes les autres.

```
monACPNonCR = PCA().fit(X)
print(monACPNonCR.explained_variance_ratio_[:4]) #le premier axe capture 99,8 % de l'inertie
monACPNonCR.components_[0] # La dernière composante de cet axe a un coefficient presque à 1
print(X)     #la dernière colonne a des valeurs à l'échelle de la centaine contrairement aux autres
```

La première valeur propre explique 99,8 % de la variance, uniquement parce que la proline domine l'inertie : ce résultat ne reflète aucune structure réelle des données. Comme pour le sommeil des mammifères vu en cours, il faut centrer et réduire les variables.

Avec les données centrées réduites on voit qu'il faut déjà 5 axes pour avoir 80 % de la variance : de nombreux variables sont décorrélées !

Question 4

Comment ont évolué les valeurs propres les plus grandes après centrage et réduction ? Combien d'axes faut-il conserver pour atteindre 95 % de la variance ? Affichez le graphique de décroissance des valeurs propres.

Correction

Après réduction, la décroissance est beaucoup plus progressive : l'axe 1 explique environ 36 % de l'inertie, l'axe 2 environ 19 %, l'axe 3 environ 11 %, et il faut environ 10 axes sur 13 pour atteindre 95 %

Si c'est pour faire un graphique 2D : il faudra s'en méfier

Si c'est pour faire de la réduction de dimension : garder 5 variables est un minimum absolu

Projection des individus (les vins)

Les coordonnées des individus dans le repère formé par les axes factoriels s'obtiennent par la méthode `transform` :

```
vinsDansNouveauRepere = monACP.transform(vinsCR)
legende_axes = [f"Axe {k+1} ({monACP.explained_variance_ratio_[k]:.1%})" for k in
range(4)]
```

Question 5

Affichez le nuage des individus projeté sur les deux premiers axes factoriels, en colorant chaque vin selon son cépage. Les trois cépages apparaissent-ils séparés ? Qu'en déduit-on ?

Correction

Les trois cépages apparaissent nettement séparés avec l'un se mélangeant un peu avec les autres. C'est une bonne surprise, l'ACP, appliquée sans utiliser l'étiquette de cépage, retrouve donc la typologie des trois cépages. On peut donc représenter la différence entre ces cépages avec seulement deux axes.

Enfin seules quelques données chimiques suffiront à caractériser ces cépages. On pourra donc réduire les mesures faites en laboratoires.

Question 6

Affichez le nuage des individus projeté sur les axes 3 et 4. Que constatez-vous ?

Correction

Sur ce plan, les trois cépages sont fortement mélangés : les axes 3 et 4, qui portent environ 18 % de l'inertie portent une information qui n'est pas caractéristique des cépages (peut être plutôt, du lieu, ou de tout autre chose).

Question 7

Réalisez l'affichage des projections sur les 3 premiers axes principaux ; la possibilité de rotation du graphique 3D facilite la lecture du nuage. Constatez vous quelque chose d'intéressant ?

Correction

Les couleurs semblent encore mieux séparées : on pourra donc envisager de projeter sur 3 axes si l'on souhaite discriminer les cépages.

Projection des variables

Question 8

Les résultats de l'analyse du nuage des variables ne sont pas directement disponibles dans Scikit-learn. Reprenez le code source du cours et affichez la projection du nuage des variables sur les deux premiers axes factoriels. Commentez.

Correction

La lecture du cercle est très instructive. À droite (axe 1 positif) la taille de sa flèche montre que les flavenoids sont presques confondus avec l'axe 1 (idem total phenols). On a une explication du sens de cet axe mais aussi le moyen de le remplacer par quelque chose de plus concret.

A gauche on voit les variables « qui s'opposent » au premières (une baisse de l'une entraîne une hausse de la valeur de l'autre) mais elles ne sont expliquées qu'à 50 % par l'axe 1 .

Le hash (hélas porté qu'à 50%) mais surtout le color intensity sont orthogonales à l'axe 2 et porte une information très importante.

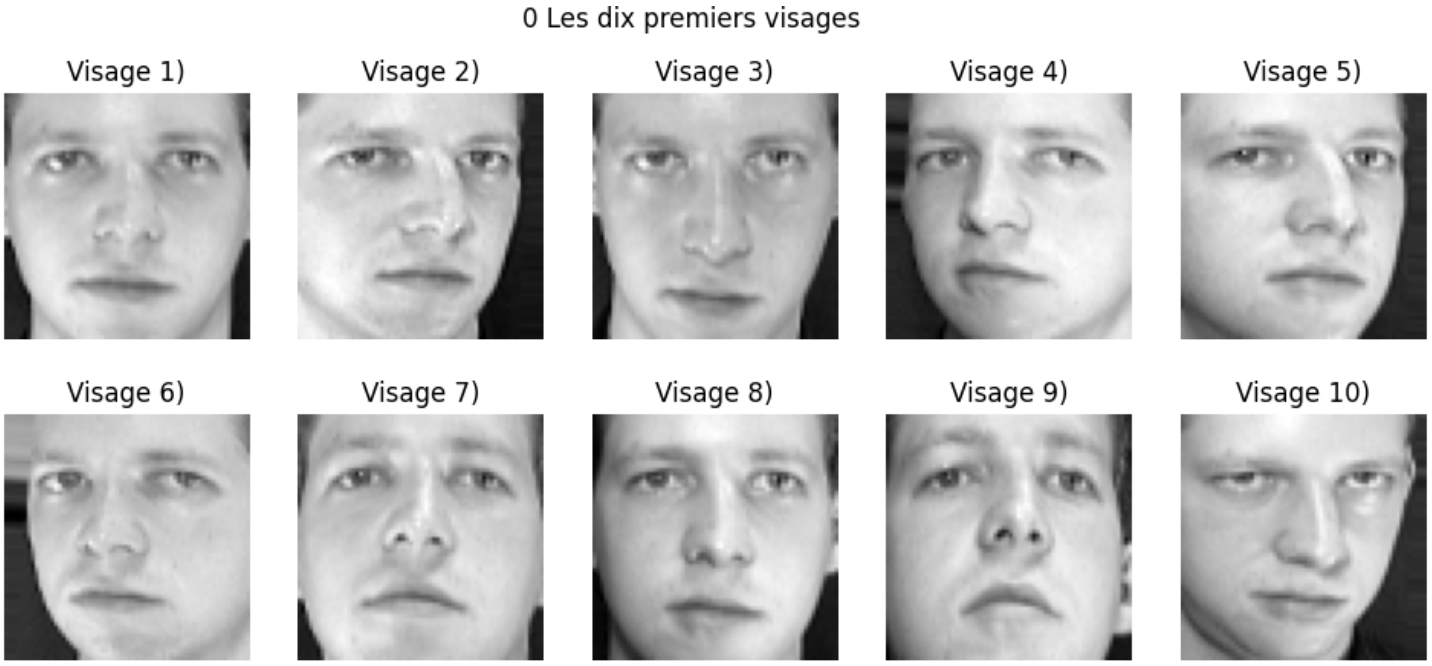
Question 9

Affichez la projection des variables sur les axes 3 et 4. Que constatez-vous par rapport aux projections sur les deux premiers axes ?

Correction

Dans ce qui précède on a vu l'intérêt d'un troisième axe, on voit qu'il est bien porté par l'alcalanité. Pour le reste la faible taille des flèches montrer que les axes sont peu explicatifs du reste. On retrouve l'idée qu'il faut de nombreux axes pour porter correctement toute l'information.

ACP sur les données « visages »



Les données utilisées dans la suite sont issues de la base de visages Olivetti (AT&T Laboratories Cambridge), distribuée avec Scikit-learn. Chaque observation est une image de visage en niveaux de gris de 64 × 64 pixels : un individu est donc décrit par 4096 variables (les intensités des pixels), toutes comprises entre 0 et 1. La base compte 400 visages, correspondant à 40 personnes photographiées chacune sous 10 poses et éclairages différents.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import fetch_olivetti_faces

visages = fetch_olivetti_faces()
X = visages.data            # (400, 4096)
personnes = visages.target  # (400,) : 40 personnes, 10 visages chacune

print("Dimensions du tableau :", X.shape)
print("Effectifs par personne :", np.bincount(personnes))
titre = "0 Les dix premiers visages"
fig, axes3 = plt.subplots(2, 5, figsize=(12, 5))
for k, ax3 in enumerate(axes3.ravel()):
    vImage = X[k].reshape(64, 64)
    ax3.imshow(vImage, cmap="gray")
    ax3.set_title(f"Visage {k+1})")
    ax3.axis("off")
fig.suptitle(titre)
```

Le premier appel à `fetch_olivetti_faces()` télécharge la base (environ 60 Mo) et la met en cache localement ; les appels suivants sont immédiats.

Question

Combien de composantes principales au maximum l'ACP peut-elle produire sur ces données ? Expliquez en fonction du nombre d'individus et du nombre de variables.

Que pensez-vous de la taille de cette base ?

Correction

L'ACP produit au plus `min(n_samples, n_features)` composantes, soit ici `min(400, 4096) = 400`. La matrice des covariances, de dimension 4096 × 4096, est de rang au plus 399 : la situation est celle dite de « grande dimension » ($p \gg n$), où le nombre de variables excède largement le nombre d'individus. C'est précisément le régime dans lequel l'ACP est la plus utile, car elle va permettre de diminuer ce nombre excessif de variables.

Évidemment un contexte « concret » comme la sécurité d'une organisation nécessiterait une base avec un bien plus grand nombre d'individus.

Centrage, réduction et calcul de l'ACP

Question

Les pixels sont tous exprimés dans la même unité (intensité entre 0 et 1). Le centrage-réduction est-il ici aussi nécessaire que pour les manchots ou les voitures ? Vérifiez l'ordre de grandeur des écarts-types des variables.

```
monScaler = StandardScaler(with_std=True).fit(X)
 #Centrer, réduire ou non, en gardant les paramètres de calcul
XCR = monScaler.transform(X)
```

`with_std` pour réduire ou non les données
`mean_` est le Vecteur des moyennes de chaque colonne de X (identique au cas par défaut)
`var_` est None (les variances ne sont pas calculées, car la réduction est désactivée)
`scale_` None (pour la même raison)

Cet exercice terminé, on fera un test avec les données réduites ou non.

Correction

On l'a vu avec les caractères, les échelles sont ici homogènes et un pixel avec un faible écart-type a bien des chances d'être inutile.

Le centrage est indispensable puisque l'ACP recherche des directions de variance autour du centre de gravité.

La réduction équilibrera le poids des zones à faible variance (arrière-plan, souvent constant) et des zones fortement variables (traits du visage.

D'un point de vue photographique, on peut penser que centrer élimine les variations de luminosité entre photos, et que réduire fait de même pour des photos au contraste différent. Mais c'est aussi un pré-traitement qui est plutôt à faire sur les photos dès la constitution du jeu de données.

Si on fait le test, au final réduire ou non change peu nos résultats or quelques valeurs de variance : il faudrait un plus grand nombre de données pour trancher le débat.

Question

A priori que peut-on dire sur le nombre d'axe à conserver ?

Affichez le graphique permettant de répondre à cette question : « Combien d'axes faut-il conserver ? »

Correction

Il faut environ 45 axes pour 80 % de variance et environ 120 axes pour 95 %

C'était prévisible, comment une poignée de valeurs pourraient-elles résumer un visage ?

Projection des individus (les visages) sur les deux premiers axes factoriels

Question

A quoi peut-on s’attendre ? Nous permettra-t-elle de différencier nos personnes ?

Cette projection suffirait-elle à différencier les personnes à couleur de peau différente ?

Affichez le nuage des individus projeté sur les deux premiers axes, chaque point étant coloré selon la personne. Les 40 personnes apparaissent-elles séparées ? Comparez avec ce qui a été observé pour les manchots.

Correction

Vu le grand nombre d’axes nécessaires il y a peu de chance que la projection change réellement quelque chose.

La différenciation de la couleur de peau ne sera pas non plus forcément faite du fait du centrage des données (la moyenne des valeurs des pixels d’une photo de personnes de couleurs différentes sera très différente).

Effectivement, aucun regroupement par personne n'est visible ici : les 40 nuages de points (un par personne) se superposent largement ce qui est logique avec seulement 27 % de l'inertie représentée.

Projection des variables : visages propres et reconstruction

Question

Le cercle des corrélations, utilisé pour les manchots et les voitures, est-il exploitable ici ? Que proposez-vous à la place pour interpréter les axes factoriels ?

Correction

Le cercle des corrélations exigerait de tracer 4096 flèches : il est illisible. Mais les variables ont ici une structure spatiale : chaque variable est un pixel, et le vecteur propre associé à un axe factoriel se met naturellement sous forme d'une image 64 × 64. Visualisés ainsi, les axes factoriels deviennent des « images propres » : la coordonnée de chaque pixel dans le vecteur propre est représentée par sa couleur et indique sa contribution à l'axe.

```
titre="3. Projection des 1797 images"
fig, axes = plt.subplots(2, 2, figsize=(13, 11), layout="constrained")
axes = axes.ravel() # liste à une dimension des axes
for i, ax in enumerate(axes):
    scatter = ax.scatter(coordonneesDansNouveauRepere[:, 2*i],
coordonneesDansNouveauRepere[:, 2*i+1],
                         c=y, cmap="tab10", s=12, alpha=0.7)
    ax.axhline(0, color="grey", lw=0.8)
    ax.axvline(0, color="grey", lw=0.8)
    ax.set_xlabel(f"Axe {2*i + 1} ({variance_expliquee[2*i]:.1%})")
    ax.set_ylabel(f"Axe {2*i + 2} ({variance_expliquee[2*i+1]:.1%})")
    ax.set_title(f"Projections sur les axes {2*i+1} et {2*i+2}")
fig.colorbar(scatter, ax=axes, label="Chiffre", shrink=0.85)
fig.suptitle(titre, fontsize=13)
```

Les premiers visages propres ressemblent à des visages fantomatiques : ils codent les variations de morphologie et d'éclairage communes à toute la base. Plus le rang de l'axe est élevé, plus le motif devient local et détaillé. L'interprétation métier des axes se fait ainsi par l'image, exactement comme le cercle des corrélations l'opérait par les corrélations variables-axes.

1Question 10

Reconstruisez un visage à partir de ses seules k premières composantes principales, pour k valant 5, 25, 50, 100 puis 200. À partir de combien d'axes le visage est-il reconnaissable ? Comment justifier la formule de reconstruction ?

2Correction

Les coordonnées factorielles complètes d'un visage suffisent : la reconstruction tronquée à k axes s'obtient en multipliant les k premières coordonnées par les k premiers vecteurs propres, puis en annulant le centrage-réduction. Aucun nouvel entraînement d'ACP n'est nécessaire.

```
titre = "4 Reconstruction d'un visage selon le nombre de composantes"
kVals = [5, 25, 50, 100, 200]
numVisage = 0
Z0 = visagesDansNouveauRepere[numVisage, :]  # coordonnées du visage choisi

fig, axes4 = plt.subplots(1, len(kVals) + 1, figsize=(15, 3))
axes4[0].imshow(X[numVisage].reshape(64, 64), cmap="gray")
axes4[0].set_title("Original")
axes4[0].axis("off")
for k, ax4 in zip(kVals, axes4[1:]):
    XrecCR = Z0[:k] @ monACP.components_[:k]              # projection tronquée
    Xrec = XrecCR * scalerAll.scale_ + scalerAll.mean_    # déstandardisation
    ax4.imshow(Xrec.reshape(64, 64), cmap="gray")
    ax4.set_title(f"{k} axes")
    ax4.axis("off")
fig.suptitle(titre)
fig.savefig(titre, dpi=150, bbox_inches="tight")
```

La formule est justifiée par la décomposition même de l'ACP : chaque visage centré-réduit est la somme exacte de ses 400 projections sur les vecteurs propres ; conserver les k premiers termes en donne la meilleure approximation au sens des moindres carrés. Dès 25 axes, le visage est généralement reconnaissable ; à 100 axes, il est difficile à distinguer de l'original. Avec 100 coordonnées au lieu de 4096 pixels, la compression est d'un facteur 40 environ — c'est le principe de la reconnaissance faciale par visages propres.

17. Classification par SVM : identification de la personne

1Question 11

En quoi le problème de classification est-il ici plus difficile que pour les manchots ? Mettez en place un protocole expérimental rigoureux, en précisant quelles transformations doivent être apprises sur le seul ensemble d'apprentissage.

2Correction

Il s'agit de distinguer 40 classes avec seulement 10 observations par classe, en grande dimension : le risque de surapprentissage est élevé. Le protocole comprend un découpage stratifié par personne, et le prétraitement (centrage-réduction, puis éventuellement l'ACP) doit être appris uniquement sur l'ensemble d'apprentissage avant d'être appliqué à l'ensemble de test, afin d'éviter toute fuite d'information du test vers le modèle.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, personnes, test_size=0.3, random_state=42, stratify=personnes)

scaler = StandardScaler().fit(X_train)
X_trainCR = scaler.transform(X_train)
X_testCR = scaler.transform(X_test)

svm = SVC(kernel="rbf", C=1.0, gamma="scale")
svm.fit(X_trainCR, y_train)
y_pred = svm.predict(X_testCR)
print("\n=== SVM sur les 4096 variables initiales ===")
print("Précision (accuracy) :", accuracy_score(y_test, y_pred))
```

Sur les 4096 pixels brutes, le SVM à noyau RBF atteint environ 85 à 90 % de précision : la quasi-totalité des visages de test sont correctement attribués à leur personne.

3Question 12

Compressez les données par ACP à 95 % de variance, puis entraînez le SVM sur les coordonnées factorielles. Comparez la précision obtenue à celle de la Question 11, puis refaites l'expérience avec seulement 25 composantes. Que constatez-vous ?

4Correction

```
acp95 = PCA(n_components=0.95).fit(X_trainCR)
Z_train = acp95.transform(X_trainCR)
Z_test = acp95.transform(X_testCR)
print("Composantes conservées (95 % de variance) :", acp95.n_components_)

svmACP = SVC(kernel="rbf", C=1.0, gamma="scale")
svmACP.fit(Z_train, y_train)
y_predACP = svmACP.predict(Z_test)
print("\n=== SVM sur les composantes principales (95 % de variance) ===")
print("Précision (accuracy) :", accuracy_score(y_test, y_predACP))

acp25 = PCA(n_components=25).fit(X_trainCR)
Z_train25 = acp25.transform(X_trainCR)
Z_test25 = acp25.transform(X_testCR)
svmACP25 = SVC(kernel="rbf", C=1.0, gamma="scale")
svmACP25.fit(Z_train25, y_train)
y_predACP25 = svmACP25.predict(Z_test25)
print("\n=== SVM sur les 25 premières composantes ===")
print("Précision (accuracy) :", accuracy_score(y_test, y_predACP25))
print("Matrice de confusion (extrait, 5 premières personnes) :")
cm = confusion_matrix(y_test, y_predACP25)
print(pd.DataFrame(cm[:5, :5],
                   index=[f"Pers. {i}" for i in range(5)],
                   columns=[f"Pers. {i}" for i in range(5)]))
```

Le SVM sur environ 120 composantes (95 % de variance) égale ou dépasse légèrement le SVM sur les 4096 pixels : en grande dimension, l'ACP supprime le bruit porté par les axes résiduels, ce qui améliore la généralisation. Avec seulement 25 composantes, la précision reste honorable (environ 80 %) pour une compression d'un facteur supérieur à 160. La matrice de confusion montre que les erreurs se concentrent sur quelques personnes dont les poses ou éclairages sont les plus atypiques.

5Question 13

Contrairement aux manchots, la visualisation des frontières de décision dans le plan des deux premiers axes serait peu lisible. Pourquoi ?

6Correction

Le plan (axe 1, axe 2) ne porte qu'environ 27 % de l'inertie et le nuage des individus y est continu, sans séparation visible entre personnes (Question 8) : tracer 40 régions de décision y serait illisible et surtout trompeur, car le classifieur décide dans un espace à 120 (ou 25) dimensions, pas dans ce plan. Lorsque la représentation factorielle sépare bien les classes — comme pour les manchots —, la projection des frontières sur le plan est informative ; ce n'est plus le cas lorsque la discrimination repose sur de nombreux axes. La validation se fait alors par les indicateurs numériques (précision, matrice de confusion) plutôt que par la visualisation.

28. Pour aller plus loin

1Question 14

Mesurez quantitativement la qualité de la compression : tracez l'erreur quadratique moyenne de reconstruction d'un visage en fonction du nombre de composantes conservées, de 1 à 200. Commentez la forme de la courbe.

2Correction

```
kMax = 200
erreurs = []
for k in range(1, kMax + 1):
    XrecCR = visagesDansNouveauRepere[:, :k] @ monACP.components_[:k]
    erreurs.append(((visagesCR - XrecCR) ** 2).mean())

fig = plt.figure(figsize=(8, 5))
ax = fig.add_subplot(111)
ax.plot(range(1, kMax + 1), erreurs, "o-", markersize=3)
```

```
ax.set_xlabel("Nombre de composantes conservées")
ax.set_ylabel("Erreur quadratique moyenne (espace centré réduit)")
ax.set_title("Qualité de la reconstruction selon le nombre d'axes")
fig.savefig("erreur-reconstruction", dpi=150, bbox_inches="tight")
```

L'erreur décroît comme la part de variance non conservée : elle chute fortement sur les premiers axes puis décroît lentement, en miroir de la courbe de variance cumulée. La totalité de l'erreur initiale correspond à la variance totale des données centrées réduites ; la reconstruction à k axes en supprime exactement la part cumulée des k premières valeurs propres. Cette courbe fournit un critère objectif pour choisir le compromis compression-fidélité, par exemple au coude de la décroissance.

3Question 15

Refaites l'analyse en remplaçant le centrage-réduction par un simple centrage (moyenne nulle, écarts-types inchangés). Les visages propres et la précision du SVM changent-ils sensiblement ? Expliquez.

4Correction

```
Xc = X - X.mean(axis=0)
monACPc = PCA().fit(Xc)
# ... même affichage des visages propres ...
```

Les résultats sont très proches : les échelles étant homogènes (Question 6), la réduction modifie peu la métrique. Les premiers visages propres restent quasi identiques et la précision du SVM varie de quelques points au plus. Ce n'est que dans le cas de variables hétérogènes — masses en grammes contre becs en millimètres, masses en livres contre accélérations en secondes — que le centrage-réduction change profondément l'analyse.

Merci à Rougier, C. *Travaux pratiques - Analyse en composantes principales*. Cnam.
https://cedric.cnam.fr/vertigo/Cours/ml/tpAcp.html

Références externes éventuelles :

- Documentation NumPy : https://numpy.org/doc/stable/
- Documentation SciPy : https://docs.scipy.org/doc/scipy/
- Documentation MatPlotLib : https://matplotlib.org/stable/users/index.html
- Site scikit-learn : https://scikit-learn.org/stable/
- Site langage Python : https://www.python.org/